MaxionBench: Conformance-Gated Decision Audits for Agentic Retrieval Memory

Anonymous Author(s)
Affiliation
Address
email

Abstract

Agentic AI systems increasingly use retrieval stores as mutable operational memory: agents insert evidence, query it under tail-latency budgets, and pay downstream context costs for retrieved passages. Existing evaluations split these concerns across static information-retrieval benchmarks and vector-database performance tests, leaving operators without an auditable way to choose a retrieval stack under deployment constraints. We present MaxionBench, a conformance-gated benchmark that evaluates retrieval infrastructure as a deployment decision audit rather than as a single leaderboard. A fixed adapter contract gates engines before scoring; three workloads test static retrieval, post-insert retrievability, and multi-evidence retrieval; and each decision card reports quality, a normalized context-cost proxy, max-row p99 latency, budget stability, objective sensitivity, and paired margins. In a single-node evaluation of FAISS CPU, LanceDB in-process, pgvector, and Qdrant, a strict 200 ms max-row p99 policy selects FAISS CPU with bge-small for all three workloads. Decision margins show that the static and streaming workloads are cost-quality near-ties resolved by tail latency, while the multi-evidence workload has both lower cost and higher evidence coverage than the nearest strict-cost alternative. Changing the objective changes selected configurations; matched audits show that an apparent aggregate pgvector quality edge is indistinguishable from FAISS; and short screening runs fail to preserve full-budget top-1 decisions on two workloads. MaxionBench therefore reports objective-conditioned, auditable deployment evidence for retrieval-memory selection.

1 Introduction

Retrieval stores are becoming operational memory for agentic AI systems. Unlike static search applications, agents repeatedly insert, update, and query evidence while operating under tail-latency and context-window constraints. A retrieval stack can therefore pass a static relevance benchmark but still fail deployment: newly inserted evidence may not be retrievable in the agent's top-k context, max-row p99 latency may exceed an interaction budget, or a high-quality configuration may impose an unacceptable downstream token footprint. We evaluate whether a conformance-gated benchmark can produce auditable experimental evidence for these retrieval-memory choices.

Existing evidence is fragmented across static relevance and systems performance. Information-retrieval benchmarks such as BEIR [Thakur et al., 2021] and HotpotQA [Yang et al., 2018] provide reusable corpora and judgments, but they do not specify adapter conformance, incremental write semantics, context-cost accounting, or deployment objectives. Vector-search and database benchmarks report latency, throughput, and recall surfaces, but those measurements are often separate from task relevance and post-insert retrievability. The gap is therefore operational: an operator needs to know which retrieval configuration is viable under a stated objective, not which engine wins one isolated metric.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

MaxionBench addresses this gap by treating benchmarking as a conformance-gated deployment decision audit. A configuration is reportable only after passing a fixed adapter contract. It is then evaluated on workloads that isolate static retrieval, streaming post-insert retrievability, and multi-evidence retrieval, and the final report exposes quality, a normalized context-cost proxy, max-row p99 latency, budget stability, objective sensitivity, and paired decision margins. These fields make the decision assumptions explicit and auditable before any configuration is treated as a winner.

The evaluation supports four findings across three named workloads: S1 (single-hop static retrieval), S2 (streaming retrieval with post-insert probes), and S3 (multi-evidence HotpotQA retrieval). First, under a strict 200 ms max-row p99 policy, FAISS CPU with `BAAI/bge-small-en-v1.5` is the policy-selected strict-latency configuration for all three workloads. Second, the selection is objective-sensitive: removing the p99 cap changes S3 multi-evidence to LanceDB in-process, and quality-first objectives choose different configurations. Third, paired audits distinguish real margins from fragile aggregate differences: S1 single-hop and S2 streaming are tail-latency tie-breaks, and the S3 multi-evidence pgvector-base aggregate edge is indistinguishable from FAISS in a 5,000-query matched audit against FAISS exact FlatIP. Fourth, B0 screening and B2 full-budget decisions diverge for S1 single-hop and S2 streaming, while S3 multi-evidence preserves top-1 despite low full-rank correlation.

The paper contributes:

1. A conformance-gated adapter protocol for retrieval infrastructure used as agentic memory, covering lifecycle operations, bulk and incremental writes, query and batch query, payload/vector updates, deletes, flush/commit behavior, index controls, and minimum stats fields.
2. Three MaxionBench workloads: S1 for single-hop static retrieval, S2 for streaming post-insert top-10 retrievability, and S3 for multi-evidence retrieval under local embedding tiers.
3. A decision-audit reporting format that separates strict-latency, cost-only/no-p99, and quality-first objectives, then reports quality, normalized context-cost proxy, max-row p99 latency, budget stability, margins, and paired audits.
4. Empirical findings showing that strict-latency, cost-only/no-p99, and quality-first objectives select different configurations; that aggregate quality edges can vanish under matched audits; and that short screening runs do not reliably preserve full-budget decisions.

The supplementary material contains the benchmark code, frozen evaluation snapshot, validation summary, conformance matrix, generated paper tables and figures, query-level paired-audit summaries, checksums, HOTPOTQA-MAXIONBENCH metadata, and a claim-evidence map.

2 Benchmark design

2.1 Systems under test

MaxionBench evaluates retrieval infrastructure for agentic memory decisions. The paper matrix runs all experiments on one CPU-only operator workstation so quality, latency, throughput, and context-cost proxy values are measured under the same hardware/runtime profile. The reportable engine set contains FAISS CPU [Johnson et al., 2019], LanceDB in-process [LanceDB, 2024], PostgreSQL with pgvector [pgvector contributors, 2024], and Qdrant [Qdrant contributors, 2024]. LanceDB service mode is listed but excluded from the paper matrix because the conformance table has no passing local service row.

MaxionBench is organized as a decision-audit pipeline: a reported winner is tied to a stated conformance gate, workload, objective, and run budget. The engine set covers four representative local operating modes: an exact in-process baseline, an embedded/local store, a PostgreSQL extension, and a service vector database. FAISS CPU serves as an exact in-process local baseline and deployment option. The adapter contract is the extension path for additional engines such as Milvus, Weaviate, Redis, Vespa, OpenSearch, or Chroma.

Every reportable engine must pass the same adapter contract before its runs can enter paper-facing tables. The contract covers lifecycle operations, bulk and incremental writes, query and batch query, vector and payload updates, delete semantics, flush or commit behavior, index controls, and minimum stats fields. The conformance suite checks CRUD correctness, post-flush visibility, filter correctness,

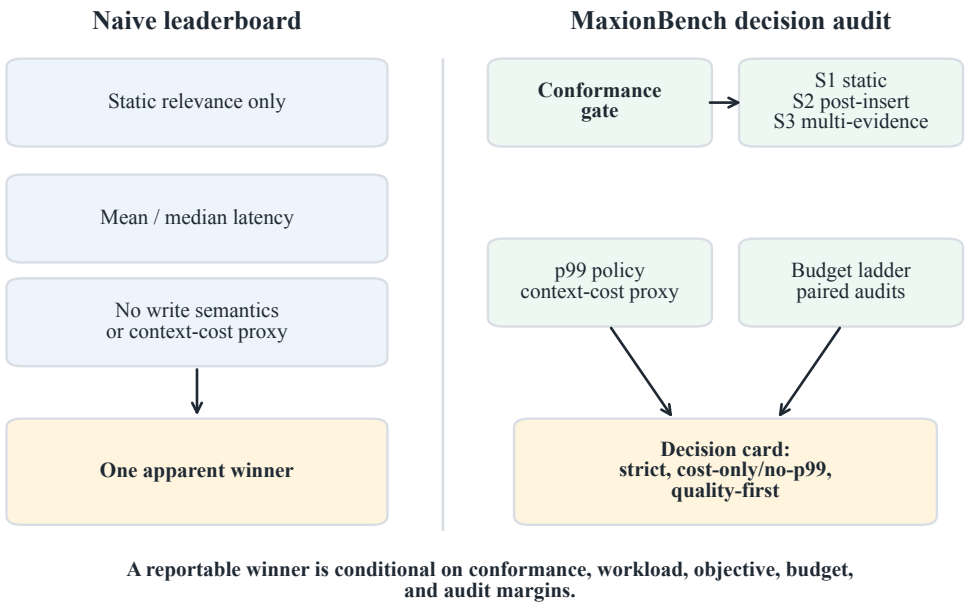

Figure 1: From leaderboard evaluation to decision audit. MaxionBench makes a deployment choice reportable only after conformance, workload evidence, p99 policy, context-cost accounting, budget stability, and paired margins are all visible.

Table 1: MaxionBench workload summary. S1 single-hop and S2 streaming counts come from the processed local BEIR-format bundles under the 50K-document paper cap; S3 multi-evidence counts come from the HotpotQA-MaxionBench manifest. Cov@10 denotes evidence_coverage@10, and all-hit@10 denotes all_evidence_hit@10.

Workload	Assets	Corpus	Queries / events	Labels	Decision role	Metrics
S1	BEIR-format SciFact + FiQA	50,000 docs	894 queries	1,685 qrels	static relevance and tail-latency baseline	nDCG@10
S2	50K BEIR background + CRAG-500 stream	50,000 background + 500 inserted stream passages	894 static queries; 500 stream events	1,685 static qrels; 500 event qrels	post-insert retrievability diagnostic under a write stream	nDCG@10; post-insert@1s/5s
S3	HOTPOTQA-MAXIONBENCH from HotpotQA dev distractor	66,635 docs	7,405 questions; 5,000-query matched-audit subset	14,810 qrels	multi-evidence and context-cost decision stress	Cov@10; all-hit@10

update semantics, delete semantics, and streaming-memory post-insert retrievability semantics. This gate prevents a benchmark table from silently mixing engines with different unsupported behavior and reports missing conformance as an explicit exclusion.

2.2 Workloads

MaxionBench uses three workloads designed to isolate retrieval-infrastructure behavior that matters to agentic applications. S1 (single-hop static retrieval) evaluates SciFact and FiQA through local BEIR-format bundles [Thakur et al., 2021, Wadden et al., 2020, Maia et al., 2018]; its primary quality metric is nDCG@10. S2 (streaming retrieval with post-insert probes) combines a deterministic 50K-passage static background from the same source family with a CRAG-500 event stream [Facebook Research, 2024]; it reports static nDCG@10 and post-insert top-10 retrievability probes at one and five seconds after insert acknowledgment. S3 (multi-evidence HotpotQA retrieval) uses HOTPOTQA-MAXIONBENCH, a frozen preprocessing of the HotpotQA dev distractor split, and measures whether the retrieved top-10 contains the supporting evidence set with evidence_coverage@10. HOTPOTQA-MAXIONBENCH contains 7,405 questions; the paper-facing S3 multi-evidence matched audit uses the configured 5,000-query subset recorded by the run configuration.

The experiments use fixed read/write-client settings so latency, throughput, and post-insert probes are comparable across engines. S1 single-hop and S3 multi-evidence evaluate read-client settings $\{1, 4, 8\}$. S2 streaming uses eight read clients and two serialized write clients, then probes each inserted event passage at $T + 1s$ and $T + 5s$, where T is the time at which the adapter's insert plus flush path returns. Each workload is evaluated with two local embedding tiers, `BAAI/bge-small-en-v1.5` and `BAAI/bge-base-en-v1.5` [Beijing Academy of Artificial Intelligence, 2023]. We write bge-small and bge-base after first mention.

2.3 Metrics and decisions

MaxionBench reports quality, tail latency, throughput, a normalized context-cost proxy, and stability across budget levels. Latency is measured on precomputed query vectors and times `adapter.query` plus top-k materialization, including service/container overhead inside the adapter call when applicable; offline embedding is excluded from latency and included only in the context-cost proxy. We rank latency by max-row p99: the maximum observed p99 across the configured B2 read-client or read/write-client rows and repeats for a candidate. We report this audit statistic with the recorded hardware/runtime profile. For S1 single-hop and S2 streaming, quality is nDCG@10. For S3 multi-evidence, quality is

$$\text{evidence_coverage@k}(q) = \frac{|E(q) \cap R_k(q)|}{|E(q)|},$$

where $E(q)$ is the gold evidence set and $R_k(q)$ is the retrieved top-k set. For S2 streaming, $\text{post_insert_hit@10}, \Delta$ is one when the newly inserted supporting passage appears in $R_{10}(q, T + \Delta)$ for $\Delta \in \{1s, 5s\}$ after the adapter's insert-plus-flush path returns. This agent-facing metric evaluates retrievability after acknowledgement; low-level index visibility is outside the metric. For S2 streaming, the strict-latency rule gates on static nDCG@10 and reports `post_insert_hit@10,1s` and `post_insert_hit@10,5s` as agent-facing write/retrieve diagnostics. We therefore interpret S2 streaming deployment choices as static-quality/cost/p99 decisions accompanied by post-insert retrievability evidence.

Within each workload and configured subset, all reportable engines evaluate the same quality query or event IDs; latency and throughput observation counts may differ because they are measured in timed windows. Paired audits join explicitly by query ID or event index. Quality differences across engines therefore reflect configured search behavior, tie handling, filtering/update semantics, and top-k materialization under the adapter contract rather than differences in the embedding model itself. In the reported MaxionBench matrix, the FAISS CPU rows have empty `index_params` and therefore use the adapter's exact flat index; recorded `hnsw_ef` sweep values are not consumed by that flat-index path.

The deployment rule is deliberately simple. A strict-latency survivor must pass the conformance gate, satisfy the workload quality floor recorded in the run configuration, and remain below the stated max-row p99 cap. For each workload, engine, and embedding tier, these survivors are ranked by the normalized context-cost proxy

$$\text{task_cost_est} = C_{\text{retrieval}} + C_{\text{embedding}} + c_{\text{llm_in}} \cdot N_{\text{retrieved_input_tokens}},$$

with lower max-row p99 and higher throughput used as tie-breaks. The report field is named `task_cost_est`, but the paper interprets it only as a normalized context-token cost proxy: it is not a cloud-dollar estimate. In the reported local run, $C_{\text{embedding}} = 0$ for offline embeddings and $c_{\text{llm_in}} = 0.15$ is a project-defined token coefficient. The strict-latency objective selects the lowest-cost survivor under the 200 ms max-row p99 cap used as the main audit policy. The report also exposes cost-only/no-p99 and quality-first objectives because those plausible operator choices can select different configurations.

Decision-card protocol. Given engines E, embedding tiers M, workloads W, budget levels B, a p99 cap τ, and objectives O, MaxionBench runs the following audit:

1. run adapter conformance checks for each engine and exclude non-conforming rows;
2. evaluate quality, max-row p99, throughput, and context-cost proxy for each workload and embedding tier;
3. rank surviving configurations under each stated objective and record strict-latency, cost-only/no-p99, and quality-first choices;
4. compute B0/B1/B2 stability statistics and run paired audits for high-risk aggregate margins;
5. emit a decision card with choices, margins, stability, conformance status, and interpretation notes.

2.4 Budget ladder

The benchmark uses three budget levels to test whether decisions are stable under shorter budgeted runs. B0 uses 10 seconds of warmup, 10 seconds of measurement, and one repeat. B1 uses 15 seconds of warmup, 30 seconds of measurement, and one repeat. B2 uses 30 seconds of warmup, 60 seconds of measurement, and two repeats. Stability is reported as Spearman rank correlation, top-1 agreement, and top-2 agreement between budget-level rankings. This design turns budget instability into a reported experimental outcome.

3 Related work

Information-retrieval benchmarks provide the relevance backbone for this work. MS MARCO [Bajaj et al., 2016], Natural Questions [Kwiatkowski et al., 2019], KILT [Petroni et al., 2021], BEIR [Thakur et al., 2021], SciFact [Wadden et al., 2020], FiQA [Maia et al., 2018], and HotpotQA [Yang et al., 2018] supply reusable corpora, tasks, and judgments, but they do not specify retrieval-engine conformance, write/retrieve probes, context-cost estimates, or deployment decision rules. MaxionBench reuses labeled retrieval evidence while adding the systems and decision-audit layer needed for local infrastructure choices.

Prior work on neural retrieval and RAG motivates the agent-facing evidence layer. Sentence-BERT [Reimers and Gurevych, 2019], DPR [Karpukhin et al., 2020], ColBERT [Khattab and Zaharia, 2020], ANCE [Xiong et al., 2021], E5 [Wang et al., 2022], and MTEB [Muennighoff et al., 2023] study representation quality and retrieval-model evaluation; REALM [Guu et al., 2020], RAG [Lewis et al., 2020], FiD [Izacard and Grave, 2021], RETRO [Borgeaud et al., 2022], RAGAS [Es et al., 2024], and RAGBench [Friel et al., 2024] study retrieval-conditioned generation or RAG evaluation. These papers clarify why retrieved evidence matters, while MaxionBench asks a lower-level deployment question: can the retrieval infrastructure surface evidence under local p99, write, and context-budget constraints?

Vector-search systems and benchmarks usually emphasize algorithmic recall, throughput, indexing configuration, and operating mode. Product quantization [Jégou et al., 2011], FLANN [Muja and Lowe, 2014], HNSW [Malkov and Yashunin, 2020], FAISS [Johnson et al., 2019], ANN-Benchmarks [Aumüller et al., 2020], the NeurIPS billion-scale ANN challenge [Simhadri et al., 2021], and vector-database suites such as VectorDBBench [Zilliz, 2023] expose important search-performance surfaces. These benchmarks are complementary to MaxionBench: they characterize ANN or product performance, whereas MaxionBench fixes an agent-facing adapter contract and ties configured search behavior to retrieval quality, post-insert top-10 retrievability, context-token cost, and deployment objectives.

Benchmark methodology and agent-memory work motivate the reporting style. MLPerf Inference [Reddi et al., 2020] emphasizes rules and reproducible system comparisons; HELM [Liang et al., 2023] emphasizes multi-metric transparency; datasheets [Gebru et al., 2021] and Croissant metadata [MLCommons, 2024] motivate explicit dataset documentation; and tail-latency work motivates high-percentile latency reporting [Dean and Barroso, 2013]. AgentBench [Liu et al., 2023], MemGPT [Packer et al., 2023], LongMemEval [Wu et al., 2024], and LoCoMo [Maharana et al., 2024] evaluate agent behavior or long-term memory. MaxionBench applies the reporting discipline to retrieval stores used as local agent memory.

4 Experiments

4.1 Evaluation validation

The reported results come from a frozen evaluation snapshot included with the supplementary material. The snapshot contains raw run records, generated figures, the HOTPOTQA-MAXIONBENCH manifest and checksums, and the conformance matrix. A strict-schema validation pass checked 72 run records and returned zero errors. Appendix Table 14 reports passing conformance and behavior-card coverage for FAISS CPU, LanceDB in-process, pgvector, and Qdrant; LanceDB service is excluded because it has no conformance row.

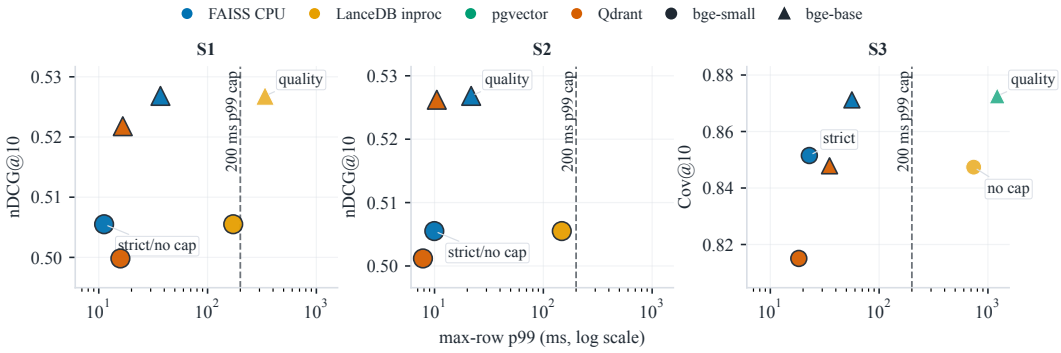

Figure 2: Decision surface for the B2 audit. The strict 200 ms max-row p99 policy selects FAISS CPU/bge-small for S1 single-hop, S2 streaming, and S3 multi-evidence, but the no-p99 and quality-first markers show that the selection is conditional on objective and policy. Each panel uses a workload-specific zoomed y-axis.

Table 2: Strict-latency deployment cards. Quality and S2 streaming post-insert top-10 retrievability report 95% confidence intervals; p99 max is the maximum observed p99 across configured B2 client rows and repeats for the selected configuration.

Workload	Engine	Emb.	Quality	post_insert_hit@10,5s	p99 max	Stability
S1 single-hop	faiss-cpu	bge-small	nDCG@10 0.506 (0.494–0.516)	–	11.2 ms	top-1 changed
S2 streaming	faiss-cpu	bge-small	nDCG@10 0.506 (0.487–0.524)	0.830 (0.805–0.852)	10.0 ms	top-1 changed
S3 multi-evidence	faiss-cpu	bge-small	Cov@10 0.852 (0.849–0.854)	–	22.8 ms	top-1 stable

The supplementary material is intended to make each paper claim traceable. It contains the benchmark code, validation summary, conformance matrix, generated tables and figures, query-level paired-audit summaries, checksums, HOTPOTQA-MAXIONBENCH Croissant-style metadata [MLCommons, 2024], and a claim-evidence map. The main results below are tied to the recorded configs, conformance gate, objectives, and budget levels.

4.2 Decision audit results

We organize results around four deployment-audit questions. First, under the default 200 ms max-row p99 policy, FAISS CPU with bge-small is the policy-selected strict-latency configuration for all three workloads. Second, this decision is objective-sensitive: removing the p99 cap or optimizing quality changes selected configurations. Third, paired audits separate clear margins from aggregate near-ties. Fourth, budget stability is workload-dependent, so short screening runs cannot always replace B2 decisions.

Table 2 is a strict-latency deployment-card summary. Under the 200 ms max-row p99 threshold, all three workloads select FAISS CPU with `BAAI/bge-small-en-v1.5`. The FAISS rows are exact FlatIP local-baseline rows, while the other rows include the configured store and adapter overheads; the table reports the decision under the recorded local runtime profile. The S1 single-hop selected row reaches nDCG@10 0.506 with a 95% confidence interval of 0.494–0.516 and max-row p99 11.2 ms. S2 streaming selects the same engine and embedding, with nDCG@10 0.506, post_insert_hit@10,5s 0.830, and max-row p99 10.0 ms. S3 multi-evidence selects FAISS CPU with `BAAI/bge-small-en-v1.5` at evidence_coverage@10 0.852 and max-row p99 22.8 ms.

S2 streaming measures agent-facing post-insert top-10 retrievability. S1 single-hop and the static-retrieval component of S2 streaming use the same 894-query BEIR-format evaluation pool under the 50K-document cap, which is why their static nDCG@10 and packed-token summaries can match. S2 streaming adds CRAG-500 write/retrieve probes after the adapter insert-plus-flush path returns. Across two B2 strict-choice repeats with 500 events each, 830 of 1,000 post-insert observations are retrieved by the 1s probe; no additional observations recover by 5s, leaving 170 of 1,000 absent at the 5s probe. Appendix Figure 6, Appendix Table 12, and Appendix Table 13 report the fixed-probe diagnostics and example event/query/document IDs.

Table 3: Deployment decisions under strict-latency, cost-only/no-p99, and quality-first objectives. Strict latency uses a 200 ms max-row p99 threshold; the S3 multi-evidence quality-first aggregate choice is treated as a matched-audit tie case in the text.

Workload	Strict p99	Cost-only/no-p99	Quality-first	Spearman B0–B2	Stability
S1 single-hop	faiss-cpu / bge-small	faiss-cpu / bge-small	lancedb-inproc / bge-base	0.79	top-1 changed
S2 streaming	faiss-cpu / bge-small	faiss-cpu / bge-small	faiss-cpu / bge-base	0.43	top-1 changed
S3 multi-evidence	faiss-cpu / bge-small	lancedb-inproc / bge-small	pgvector / bge-base (matched tie)	0.31	top-1 stable

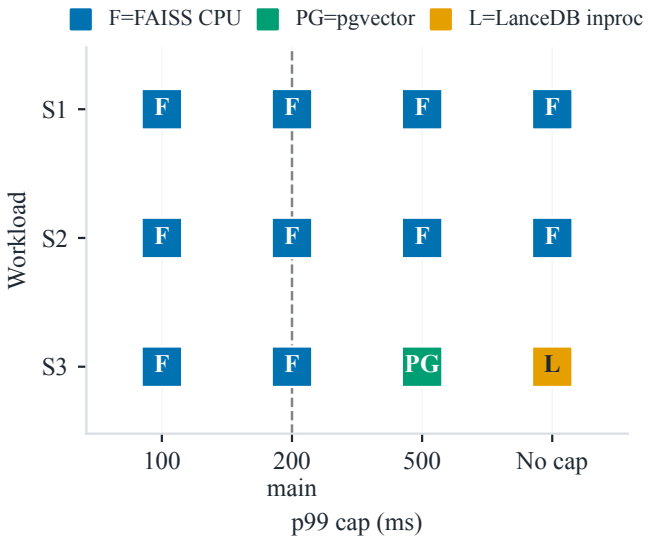

Figure 3: S3 multi-evidence is sensitive to relaxing the p99 policy, while S1 single-hop and S2 streaming keep the same policy-selected engine across the reported thresholds. This threshold sweep shows how MaxionBench reports decisions under stated local policies.

The benchmark-design ablation is reported in Appendix Table 9. Its main implication is that removing any audit component changes the interpretation: the p99 gate separates strict-latency decisions from cost-only/no-p99 choices, objective separation prevents quality-first rows from being conflated with strict-latency rows, budget reporting exposes unstable S1 single-hop and S2 streaming screening runs, paired audits prevent aggregate near-ties from being overclaimed, and conformance filtering keeps unsupported service rows out of paper-facing tables.

Table 3 and Figure 2 show objective sensitivity directly. The strict-latency and cost-only/no-p99 objectives agree for S1 single-hop and S2 streaming, but S3 multi-evidence shifts to LanceDB in-process with `BAAI/bge-small-en-v1.5` when the p99 cap is removed. Quality-first decisions differ more strongly: LanceDB in-process with `BAAI/bge-base-en-v1.5` leads S1 single-hop, FAISS CPU with `BAAI/bge-base-en-v1.5` leads S2 streaming, and the aggregate quality-first rule selects pgvector with `BAAI/bge-base-en-v1.5` for S3 multi-evidence. The threshold sweep makes the 200 ms policy explicit and auditable.

The S3 multi-evidence quality-first margin is the highest-risk aggregate claim, so we audit it with matched queries. Using all B2 rows, pgvector-base exceeds FAISS-base by only 0.00016 evidence_coverage@10 with an aggregate-row bootstrap interval of -0.00280 to 0.00304. The matched audit in Figure 4 removes even that apparent edge: pgvector IVF32 and IVF64 are each 0.0001 evidence_coverage@10 below FAISS exact FlatIP on the 5,000-query matched-audit subset, with paired intervals including zero. Appendix Table 17 gives the numeric rows, and Appendix Table 18 records the stricter binary all_evidence_hit@10 audit.

Appendix Table 11 makes the strict-latency decision margins explicit. For S1 single-hop and S2 streaming, FAISS CPU with `BAAI/bge-small-en-v1.5` matches the closest LanceDB in-process candidate on relevance quality and cost, so the deployment rule resolves the tie by lower max-row p99 and higher throughput. For S3 multi-evidence, Qdrant with `BAAI/bge-small-en-v1.5` is the nearest strict-cost alternative but is more expensive and lower quality, even though its max-row p99 is

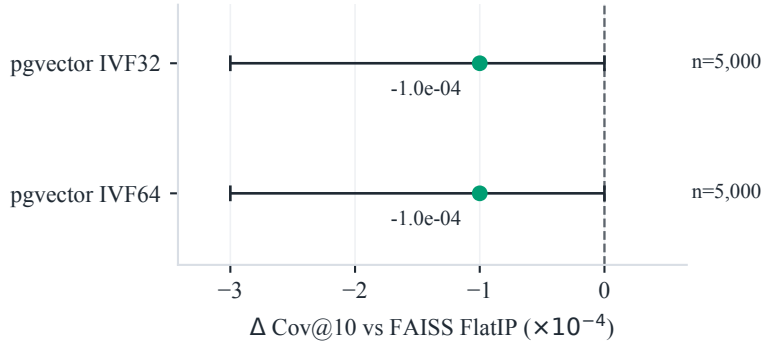

Figure 4: Matched S3 multi-evidence paired audit. Against FAISS exact FlatIP on the same 5,000 bge-base queries, pgvector IVF32 and IVF64 have paired evidence_coverage@10 deltas whose 95% intervals include zero, so the aggregate pgvector quality-first row is indistinguishable under matched queries.

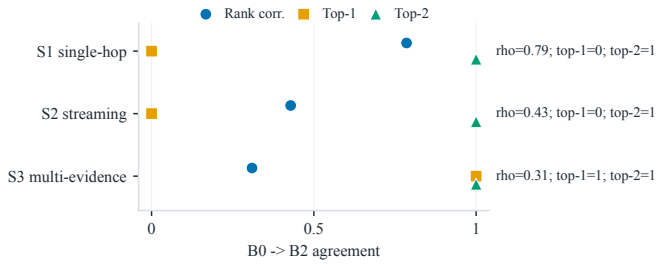

Figure 5: Budget stability is workload-dependent. B0 screening fails to preserve B2 top-1 choices for S1 single-hop and S2 streaming, while S3 multi-evidence preserves top-1 despite low full-rank Spearman correlation.

4.6 ms lower in the repeat-aware table. The highest-quality strict-latency S3 multi-evidence option remains FAISS CPU with `BAAI/bge-base-en-v1.5`, which improves evidence_coverage@10 by 0.020 over `BAAI/bge-small-en-v1.5` but increases the normalized context-cost proxy by 0.704 and max-row p99 by 33.4 ms on the B2 audit rows.

The normalized context-cost proxy is stable under the coefficient sensitivity tested in the supplementary analysis. Appendix Table 5 rescales only $c_{\text{llm_in}}$ from $0.1\times$ to $10\times$ and leaves all strict choices unchanged: S1 single-hop and S2 streaming have identical retrieved-token counts for the strict choice and nearest strict-cost competitor, while S3 multi-evidence FAISS/bge-small retrieves slightly fewer tokens than Qdrant/bge-small. Appendix Table 6 reports p50, p95, mean p99, and the min–max p99 range over B2 run rows for the recorded hardware/runtime profile.

The budget analysis measures how much evidence is enough. B0-to-B2 top-1 agreement is zero for S1 single-hop and S2 streaming, with Spearman correlations 0.79 and 0.43, respectively. S3 multi-evidence has top-1 agreement of one but Spearman correlation 0.31, meaning that the selected strict-latency row is stable while lower-ranked alternatives remain noisy. The resulting decision card includes the budget level and stability statistics used to reach each recommendation.

4.3 Targeted replication checks

Targeted same-machine reruns support the decision-level conclusions and quantify tail-latency variation. Appendix Table 8 shows that the strict FAISS CPU/bge-small choices reproduce the relevant quality values and remain under the 200 ms policy; the S3 multi-evidence bge-base rerun and Figure 4 support the objective-sensitivity story; and Appendix Table 20 reports a larger FAISS-versus-Qdrant S2 streaming paired check that finds no Qdrant quality or post-insert retrievability advantage. Exact repeat numbers remain in the appendix to keep the observed p99 variation visible.

5 Experimental boundaries and broader impact

MaxionBench reports a fixed, reproducible CPU-only evaluation setting for retrieval-memory decisions. The experiments cover FAISS CPU, LanceDB in-process, pgvector, and Qdrant under the adapter settings recorded in the supplementary material; distributed retrieval, GPU indexing, cloud variability, second-machine portability, and multi-tenant workloads are natural extensions of the same conformance-gated protocol. The S3 multi-evidence workload uses HOTPOTQA-MAXIONBENCH, a frozen paragraph-level HotpotQA preprocessing, and the S2 streaming workload measures top-k retrievability after inserts. Engine-level quality differences are configured-search differences under the MaxionBench adapter settings. The S3 multi-evidence paired audit uses FAISS exact FlatIP as the bge-base reference, and future service-engine sweeps can vary higher probe and ef settings within the same adapter contract.

The uncertainty estimates describe the recorded benchmark runs. Confidence intervals are query- or event-level summaries, and max-row p99 latency is reported with the hardware/runtime profile that produced it. S1 single-hop and S2 streaming change top-1 recommendations between B0 and B2, while S3 multi-evidence keeps the same top-1 recommendation but has low full-rank correlation; these budget effects are part of the experimental result. The broader impact is methodological: a reproducible benchmark can help operators choose lower-cost, lower-latency retrieval infrastructure while keeping quality, post-insert retrievability, objective choice, budget level, and paired margins visible.

6 Conclusion

MaxionBench frames retrieval infrastructure for agentic AI as an auditable deployment decision rather than a leaderboard. The key idea is to report a configuration only after it passes a fixed adapter contract, then expose workload quality, post-insert top-10 retrievability, max-row p99 latency, normalized context-cost proxy, budget stability, objective sensitivity, and paired margins in one decision card. The experimental evidence shows why that audit matters: the strict 200 ms policy selects FAISS CPU/bge-small for S1 single-hop, S2 streaming, and S3 multi-evidence, but alternative objectives change decisions, S1 single-hop and S2 streaming are tail-latency tie-breaks, the S3 multi-evidence pgvector aggregate edge is indistinguishable under matched queries, and short B0 runs do not preserve all B2 decisions. The operational conclusion is that retrieval-memory deployment choices are more credible when their conformance, objective, budget, and decision margins are explicit.

References

Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems*, 87:101374, 2020.

Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, Mir Rosenberg, Xia Song, Alina Stoica, Saurabh Tiwary, and Tong Wang. MS MARCO: A human generated MAchine reading COmprehension dataset, 2016. URL `https://arxiv.org/abs/1611.09268`.

Beijing Academy of Artificial Intelligence. BAAI general embedding models. `https://huggingface.co/BAAI`, 2023. Model identifiers bge-small-en-v1.5 and bge-base-en-v1.5.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 2206–2240. PMLR, 2022. URL `https://proceedings.mlr.press/v162/borgeaud22a.html`.

Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024.

Facebook Research. CRAG: Comprehensive rag benchmark. `https://github.com/facebookresearch/CRAG`, 2024. Dataset source used for the CRAG-500 event stream.

Robert Friel, Masha Belyi, and Atindriyo Sanyal. RAGBench: Explainable benchmark for retrieval-augmented generation systems, 2024.

Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12): 86–92, 2021.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3929–3938. PMLR, 2020. URL `https://proceedings.mlr.press/v119/guu20a.html`.

Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 874–880, 2021. doi: 10.18653/v1/2021.eacl-main.74. URL `https://aclanthology.org/2021.eacl-main.74/`.

Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011. doi: 10.1109/TPAMI.2010.57. URL `https://doi.org/10.1109/TPAMI.2010.57`.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 2019.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781, 2020. doi: 10.18653/v1/2020.emnlp-main.550. URL `https://aclanthology.org/2020.emnlp-main.550/`.

Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020. doi: 10.1145/3397271.3401075. URL `https://doi.org/10.1145/3397271.3401075`.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL https://aclanthology.org/Q19-1026/.

LanceDB. LanceDB: Developer-friendly vector database. https://lancedb.com/, 2024.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020. URL https://arxiv.org/abs/2005.11401.

Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents, 2023.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents, 2024.

Macedo Maia, Siegfried Handschuh, André Freitas, Brian Davis, Ross McDermott, Manel Zarrouk, and Alexandra Balahur. WWW'18 open challenge: Financial opinion mining and question answering. In *Companion Proceedings of the Web Conference*, 2018.

Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. doi: 10.1109/TPAMI.2018.2889473. URL https://doi.org/10.1109/TPAMI.2018.2889473.

MLCommons. Croissant format specification, version 1.0. https://docs.mlcommons.org/croissant/docs/croissant-spec.html, 2024. Published March 1, 2024.

Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, 2023.

Marius Muja and David G. Lowe. Scalable nearest neighbor algorithms for high dimensional data. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 36(11):2227–2240, 2014. doi: 10.1109/TPAMI.2014.2321376. URL https://doi.org/10.1109/TPAMI.2014.2321376.

Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023.

Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, Vassilis Plachouras, Tim Rocktäschel, and Sebastian Riedel. KILT: a benchmark for knowledge intensive language tasks. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2523–2544, 2021. doi: 10.18653/v1/2021.naacl-main.200. URL https://aclanthology.org/2021.naacl-main.200/.

pgvector contributors. pgvector: Open-source vector similarity search for PostgreSQL. https://github.com/pgvector/pgvector, 2024.

Qdrant contributors. Qdrant: Vector database and similarity search engine. https://qdrant.tech/, 2024.

Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, et al. MLPerf inference benchmark. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture*, 2020.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3982–3992, 2019. doi: 10.18653/v1/D19-1410. URL https://aclanthology.org/D19-1410/.

Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamy, Gopal Srinivasa, Suhas Jayaram Subramanya, and Jingdong Wang. Results of the NeurIPS'21 challenge on billion-scale approximate nearest neighbor search. In *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track*, 2021.

Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Advances in Neural Information Processing Systems*, 2021.

David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. SciFact: A dataset for fact verification of scientific claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020.

Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training, 2022. URL https://arxiv.org/abs/2212.03533.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory, 2024.

Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. Approximate nearest neighbor negative contrastive learning for dense text retrieval. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=zeFrfgyZln.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018.

Zilliz. VectorDBBench: A benchmark tool for vector databases. https://github.com/zilliztech/VectorDBBench, 2023.

A Reproducibility details

The supplementary material provides the workflow instructions used to reproduce the benchmark setup, data preparation, B0–B2 runs, final report generation, and strict-schema validation. The frozen evaluation snapshot used by this manuscript contains raw run records, generated paper figures, HOTPOTQA-MAXIONBENCH metadata, and conformance evidence. The validation summary reports `"pass": true`, `"error_count": 0`, and `"run_dirs_checked": 72`.

The validated MaxionBench matrix contains 72 run-result tables, including 24 B2 result tables with matching run metadata. The public submission records a CPU-only local runtime profile and omits machine-specific identifiers. The reported matrix uses CPU rows throughout. HOTPOTQA-MAXIONBENCH contains 66,635 documents, 7,405 questions, and 14,810 qrels; the verified manifest fields are `doc_count=66635`, `question_count=7405`, and `qrel_count=14810`. The S3 multi-evidence matched audit uses the configured 5,000-query subset cap recorded in the run configuration, while the full HOTPOTQA-MAXIONBENCH manifest remains part of the supplement. The HotpotQA-MaxionBench checksum manifest stores SHA-256 checksums for the corpus, queries, qrels, metadata, and manifest.

The supplementary package also includes an artifact card, a consistency verifier, and Croissant-style metadata for HOTPOTQA-MAXIONBENCH. The verifier checks that the manuscript, evaluation manifest, paper-facing tables, paired-audit summaries, and metadata files are present and internally consistent; the Croissant metadata records the corpus, query, and qrel files, their intended evaluation use, and the license/provenance boundary inherited from HotpotQA.

B Audit tables

Table 4: Normalized context-cost proxy terms and deterministic value sources. The reported proxy is $task_cost_est = rhu_h/\max(measured_requests, 1) + 0.0 + c_llm_in \times avg_retrieved_input_tokens$, with $c_llm_in = 0.15$ in the resolved run configs and metadata; it is not a cloud-dollar estimate.

Term	Meaning	Unit	Value source
C_retrieval	Per-query retrieval cost estimate	project-defined normalized cost	retrieval_cost_est = rhu_h / max(measured_requests, 1)
C_embedding	Per-query embedding cost estimate	project-defined normalized cost	embedding_cost_est = 0.0 for offline local embeddings
c_llm_in	Input-token cost coefficient	project-defined normalized cost per token	0.15 from RunConfig.c_llm_in
Nretrieved_input_tokens	Retrieved context tokens packed downstream	tokens/query	avg_retrieved_input_tokens measured from top-k outputs
task_cost_est	C_retrieval + C_embedding + c_llm_in x Nretrieved_input_tokens	normalized context-cost/query	retrieval_cost_est + embedding_cost_est + llm_context_cost_est

Table 5: Strict-choice cost sensitivity to input-token cost multipliers. The 0.1×, 1×, and 10× rows rescale only c_llm_in; strict choices do not change because the strict choice and nearest strict-cost competitor have identical S1 single-hop/S2 streaming token counts and lower S3 multi-evidence token count for FAISS CPU.

Workload	Role	Candidate	Mult.	Tokens	Cost	Changes?	Selected
S1 single-hop	strict choice	faiss-cpu / bge-small	0.1	1752	26.276	no	faiss-cpu / bge-small
S1 single-hop	nearest strict-cost competitor	lancedb-inproc / bge-small	0.1	1752	26.276	no	faiss-cpu / bge-small
S1 single-hop	strict choice	faiss-cpu / bge-small	1.0	1752	262.759	no	faiss-cpu / bge-small
S1 single-hop	nearest strict-cost competitor	lancedb-inproc / bge-small	1.0	1752	262.759	no	faiss-cpu / bge-small
S1 single-hop	strict choice	faiss-cpu / bge-small	10.0	1752	2627.592	no	faiss-cpu / bge-small
S1 single-hop	nearest strict-cost competitor	lancedb-inproc / bge-small	10.0	1752	2627.592	no	faiss-cpu / bge-small
S2 streaming	strict choice	faiss-cpu / bge-small	0.1	1752	26.276	no	faiss-cpu / bge-small
S2 streaming	nearest strict-cost competitor	lancedb-inproc / bge-small	0.1	1752	26.276	no	faiss-cpu / bge-small
S2 streaming	strict choice	faiss-cpu / bge-small	1.0	1752	262.759	no	faiss-cpu / bge-small
S2 streaming	nearest strict-cost competitor	lancedb-inproc / bge-small	1.0	1752	262.759	no	faiss-cpu / bge-small
S2 streaming	strict choice	faiss-cpu / bge-small	10.0	1752	2627.592	no	faiss-cpu / bge-small
S2 streaming	nearest strict-cost competitor	lancedb-inproc / bge-small	10.0	1752	2627.592	no	faiss-cpu / bge-small
S3 multi-evidence	strict choice	faiss-cpu / bge-small	0.1	862	12.932	no	faiss-cpu / bge-small
S3 multi-evidence	nearest strict-cost competitor	qdrant / bge-small	0.1	866	12.984	no	faiss-cpu / bge-small
S3 multi-evidence	strict choice	faiss-cpu / bge-small	1.0	862	129.325	no	faiss-cpu / bge-small
S3 multi-evidence	nearest strict-cost competitor	qdrant / bge-small	1.0	866	129.835	no	faiss-cpu / bge-small
S3 multi-evidence	strict choice	faiss-cpu / bge-small	10.0	862	1293.249	no	faiss-cpu / bge-small
S3 multi-evidence	nearest strict-cost competitor	qdrant / bge-small	10.0	866	1298.351	no	faiss-cpu / bge-small

Table 6: Latency distributions for strict choices and objective-sensitivity rows. Boundary: precomputed query vector; timed adapter.query plus top-k materialization, including service/container overhead inside the adapter call when applicable; offline embedding is excluded. The p99 range is the min–max over B2 run rows for the listed configuration.

Workload	Role	Engine	Embedding	R/W clients	p50	p95	p99	p99 range	Rows/reps	Obs.
S1 single-hop	strict choice	faiss-cpu	bge-small	1 / 0, 4 / 0, 8 / 0	3.8	5.9	6.3	1.5–11.2	6/2	5364
S1 single-hop	cost-only/no-p99 choice	faiss-cpu	bge-small	1 / 0, 4 / 0, 8 / 0	3.8	5.9	6.3	1.5–11.2	6/2	5364
S1 single-hop	quality-first choice	lancedb-inproc	bge-base	1 / 0, 4 / 0, 8 / 0	125.3	146.1	171.9	24.6–337.4	6/2	5364
S2 streaming	strict choice	faiss-cpu	bge-small	8 / 2	5.7	9.8	10.0	9.9–10.0	2/2	1788
S2 streaming	cost-only/no-p99 choice	faiss-cpu	bge-small	8 / 2	5.7	9.8	10.0	9.9–10.0	2/2	1788
S2 streaming	quality-first choice	faiss-cpu	bge-base	8 / 2	12.0	21.1	21.7	21.4–21.7	2/2	1788
S3 multi-evidence	strict choice	faiss-cpu	bge-small	1 / 0, 4 / 0, 8 / 0	5.3	8.7	11.6	2.4–22.8	6/2	30000
S3 multi-evidence	cost-only/no-p99 choice	lancedb-inproc	bge-small	1 / 0, 4 / 0, 8 / 0	203.9	276.7	396.3	54.8–739.6	6/2	7469
S3 multi-evidence	quality-first choice	pgvector	bge-base	1 / 0, 4 / 0, 8 / 0	350.2	540.5	715.2	268.3–1217.3	6/2	2827

Table 7: Latency distributions for nearest strict-cost competitors. Boundary: precomputed query vector; timed adapter.query plus top-k materialization, including service/container overhead inside the adapter call when applicable; offline embedding is excluded. The p99 range is the min–max over B2 run rows for the listed configuration.

Workload	Role	Engine	Embedding	R/W clients	p50	p95	p99	p99 range	Rows/reps	Obs.
S1 single-hop	nearest strict-cost competitor	lancedb-inproc	bge-small	1 / 0, 4 / 0, 8 / 0	66.1	77.9	96.4	17.1–172.1	6/2	5364
S2 streaming	nearest strict-cost competitor	lancedb-inproc	bge-small	8 / 2	78.3	91.9	147.8	96.5–147.8	2/2	1788
S3 multi-evidence	nearest strict-cost competitor	qdrant	bge-small	1 / 0, 4 / 0, 8 / 0	7.2	9.4	11.0	6.1–18.3	6/2	30000

Table 8: Same-machine repeat audit for the strict FAISS CPU/bge-small choices. The p99 range summarizes completed local repeat/configured-client rows under the recorded hardware/runtime profile.

Workload	Rows	Repeats	Quality	post@5s	p99 min/med/max ms	Errors
S1 single-hop	9	3	nDCG@10=0.506	–	2.4/8.9/27.7	0
S2 streaming	2	2	nDCG@10=0.506	0.840	21.4/47.8/74.1	0
S3 multi-evidence	9	3	Cov@10=0.852	–	3.6/9.6/20.6	0

Table 9: Protocol components whose omission changes or weakens deployment conclusions.

Missing component	Wrong conclusion caused by omission	Evidence
p99 latency gate	S3 multi-evidence cost-only/no-p99 choice shifts to lancedb-inproc/bge-small instead of faiss-cpu/bge-small	decision table and threshold-sensitivity panel
Objective separation	Quality-first decisions differ from strict-latency decisions on S1 single-hop, S2 streaming, and S3 multi-evidence	decision table
Budget ladder	B0 screening fails to preserve B2 top-1 decisions for S1 single-hop and S2 streaming	budget-stability figure and decision table
Paired S3 multi-evidence audit	pgvector/bge-base appears quality-first for S3 multi-evidence, but the matched 5,000-query audit removes the substantive margin	S3 multi-evidence paired audit table
Paired S2 streaming competitor audit	Paired check finds no Qdrant quality or post-insert retrievability advantage	S2 streaming competitor audit table
Conformance gate	Engines enter paper-facing result tables only after passing local conformance rows	benchmark setup, support table, and evaluation validation

Table 10: B2 decision-surface rows for strict survivors and objective winners. Rows include all strict-p99 survivors plus cost-only/no-p99 and quality-first choices when they are outside the strict survivor set; p99 max is the maximum observed B2 p99 across configured read-client rows and repeats.

Workload	Role	Engine	Emb.	Quality	post@5s	Cost proxy	p99 max	Strict pass
S1 single-hop	strict choice; cost-only/no-p99	faiss-cpu	bge-small	0.506	–	262.759	11.2	yes
S1 single-hop	strict survivor	lancedb-inproc	bge-small	0.506	–	262.759	172.1	yes
S1 single-hop	strict survivor	qdrant	bge-small	0.500	–	263.311	15.8	yes
S1 single-hop	strict survivor	faiss-cpu	bge-base	0.527	–	282.561	36.9	yes
S1 single-hop	strict survivor	qdrant	bge-base	0.522	–	283.320	16.6	yes
S1 single-hop	quality-first	lancedb-inproc	bge-base	0.527	–	282.561	337.4	no
S2 streaming	strict choice; cost-only/no-p99	faiss-cpu	bge-small	0.506	0.830	262.759	10.0	yes
S2 streaming	strict survivor	lancedb-inproc	bge-small	0.506	0.830	262.759	147.8	yes
S2 streaming	strict survivor	qdrant	bge-small	0.501	0.830	263.114	7.8	yes
S2 streaming	strict survivor; quality-first	faiss-cpu	bge-base	0.527	0.846	282.561	21.7	yes
S2 streaming	strict survivor	qdrant	bge-base	0.526	0.846	282.861	10.5	yes
S3 multi-evidence	strict choice	faiss-cpu	bge-small	0.852	–	129.325	22.8	yes
S3 multi-evidence	strict survivor	qdrant	bge-small	0.815	–	129.835	18.3	yes
S3 multi-evidence	strict survivor	faiss-cpu	bge-base	0.871	–	130.029	56.2	yes
S3 multi-evidence	strict survivor	qdrant	bge-base	0.848	–	130.474	35.0	yes
S3 multi-evidence	cost-only/no-p99	lancedb-inproc	bge-small	0.847	–	128.492	739.6	no
S3 multi-evidence	quality-first	pgvector	bge-base	0.873	–	131.113	1217.3	no

Table 11: Strict-latency decision margins against the nearest strict-cost candidate. Δ values are candidate minus strict choice; Δp99 is computed from Table 7's p99-max values.

Workload	Strict choice	Next strict candidate	Metric	Δquality	Δcost	Δp99	Interpretation
S1 single-hop	faiss-cpu / BAAI/bge-small-en-v1.5	lancedb-inproc / BAAI/bge-small-en-v1.5	nDCG@10	+0.000	+0.000	+160.9 ms	cost/quality tie; p99 tie-break
S2 streaming	faiss-cpu / BAAI/bge-small-en-v1.5	lancedb-inproc / BAAI/bge-small-en-v1.5	nDCG@10	+0.000	+0.000	+137.8 ms	cost/quality tie; p99 tie-break
S3 multi-evidence	faiss-cpu / BAAI/bge-small-en-v1.5	qdrant / BAAI/bge-small-en-v1.5	Cov@10	-0.036	+0.510	-4.6 ms	candidate is more expensive, lower quality, slightly faster

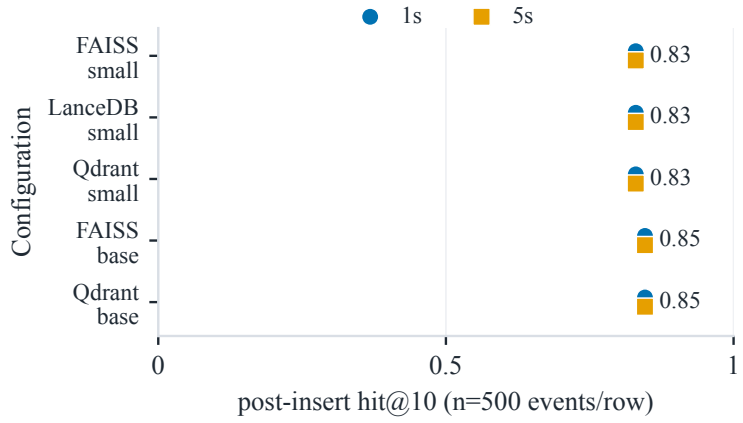

Figure 6: S2 streaming post-insert top-10 retrievability for B2 strict survivors. One- and five-second probes overlap in the validated rows, showing no additional recoveries between the fixed probes.

Table 12: S2 streaming write/retrieve diagnostics for B2 strict survivors and the quality-first row. Probe-window p95 is derived from the fixed 1s/5s post-insert probes after adapter insert plus flush returns and reports agent-facing probe timing.

Role	Engine	Emb.	nDCG@10	post@1s	post@5s	probe-window p95	Events	Errors
strict choice	faiss-cpu	bge-small	0.506	0.830	0.830	5000.0 ms	500	0
strict survivor	lancedb-inproc	bge-small	0.506	0.830	0.830	5000.0 ms	500	0
strict survivor	qdrant	bge-base	0.526	0.846	0.846	5000.0 ms	500	0
strict survivor	qdrant	bge-small	0.501	0.830	0.830	5000.0 ms	500	0
strict survivor; quality-first	faiss-cpu	bge-base	0.527	0.846	0.846	5000.0 ms	500	0

Table 13: S2 streaming strict-choice post-insert example outcomes. Counts are freshness observations for FAISS CPU/bge-small B2 rows; examples show IDs only because the observation schema records event/query/document IDs and hit flags, not ranked-list text.

Outcome	Count	Rate	Example event (vis.)	Query	Target doc	Hits 1s/5s	Interpretation
retrieved by 1s	830	0.830	0 (<0.1 ms)	7bb29eb4...3c8962	7bb29eb4..._p0_c0	1/1	target passage appears in top-10 at the first fixed probe
recovered between 1s and 5s	0	0.000	–	–	–	–/–	no strict-choice events recovered only at the later probe
missed by 5s	170	0.170	6 (5000.0 ms)	31f7aaad...4ba2ac	31f7aaad..._p0_c0	0/0	target passage absent from top-10 at the final fixed probe; outcome is censored at 5s
inconsistent hit flags	0	0.000	–	–	–	–/–	hit flags do not form a monotone probe outcome

Table 14: Conformance and behavior-card support for paper-facing reportability.

Engine	Behavior card	Conformance	Reported?	Exclusion reason
faiss-cpu	faiss_cpu.md	pass	yes	
lancedb-inproc	lancedb.md	pass	yes	
lancedb-service	lancedb.md		no	missing conformance row
pgvector	pgvector.md	pass	yes	
qdrant	qdrant.md	pass	yes	

Table 15: Engine version and run-mode metadata for the reported matrix.

Engine	Mode	Version	Metric	Dim.	Reported?
faiss-cpu	in-process	faiss-cpu 1.8.0.post1	ip	384, 768	yes
lancedb-inproc	in-process	lancedb 0.25.3	ip	384, 768	yes
lancedb-service	service API	not reported; no passing local lancedb-service conformance row	ip		no
pgvector	server database	pgvector 0.8.2 on PostgreSQL 16; image pgvector/pgvector:0.8.2-pg16-trixie	ip	384, 768	yes
qdrant	server API	Qdrant server v1.17.1; image qdrant/qdrant:v1.17.1; requests 2.31.0	ip	384, 768	yes

Table 16: Engine index/search and flush semantics for the reported matrix.

Engine	Index/search	Process model	Flush/commit
faiss-cpu	index_type=flat (exact IndexFlatIP); search_sweep recorded but ignored by flat index; top_k=10	Python process-local adapter	staged writes become visible after flush_or_commit rebuild/update
lancedb-inproc	default LanceDB table search; adapter does not create a secondary index; search_sweep recorded but not consumed by adapter; top_k=10	Python in-process library with local LanceDB URI	staged writes become visible after flush_or_commit
lancedb-service	default adapter configuration	external HTTP service contract	depends on service implementation; no passing local row
pgvector	index_method=ivfflat (adapter default); search_sweep=[{"ivfflat_probes": 32}, {"ivfflat_probes": 64}]; top_k=10	PostgreSQL server plus Python adapter	writer transaction commit; readers observe committed state
qdrant	search_sweep=[{"hnsw_ef": 32}, {"hnsw_ef": 64}]; top_k=10	Qdrant server plus Python adapter	upsert/update/delete requests use wait=true

Table 17: Matched-query S3 multi-evidence quality audit for the quality-first margin. Each row evaluates the same 5,000-query HOTPOTQA-MAXIONBENCH matched-audit subset once with `BAAI/bge-base-en-v1.5` and `clients_read=4`. Paired deltas are against FAISS exact FlatIP in evidence_coverage@10; intervals are paired bootstrap 95% intervals.

Engine / search	evidence_coverage@10	p99 (ms)	QPS	Paired Δ evidence_coverage@10
FAISS CPU / exact FlatIP	0.8712	17.6	247.6	reference
pgvector / IVF32	0.8711	306.0	15.2	-0.0001 [-0.0003, 0.0000]
pgvector / IVF64	0.8711	346.2	14.6	-0.0001 [-0.0003, 0.0000]

Table 18: S3 multi-evidence all-evidence hit@10 audit rows. All-hit observation rows count only rows used to compute the stricter binary all-hit metric; rows with Cov@10 and zero all-hit observations inherit Cov@10 from the decision table but have no all-hit audit sample.

Role	Engine	Embedding	Cov@10	All-hit@10	All-hit obs. rows
strict choice	faiss-cpu	bge-small	0.852	0.722	61896
quality-first row	pgvector	bge-base	0.873	–	0
cost-only/no-p99 row	lancedb-inproc	bge-small	0.847	–	0
high-quality strict-latency row	faiss-cpu	bge-base	0.871	–	0
next strict-cost candidate	qdrant	bge-small	0.815	–	0

Table 19: Matched S1 single-hop/S2 streaming FAISS CPU versus LanceDB in-process audit with `BAAI/bge-small-en-v1.5`. Paired deltas are LanceDB minus FAISS.

Workload	Comparison	Quality obs.	Δ quality	95% CI	Post-insert events	Δ post_insert_hit@10,5s	p99 direction	Conclusion
S1 single-hop	LanceDB − FAISS	1788	0.0000	[0.0000, 0.0000]	–	–	FAISS lower p99	tail-latency tie-break; paired quality interval contains zero
S2 streaming	LanceDB − FAISS	1788	0.0000	[0.0000, 0.0000]	40	0.0000	FAISS lower p99	tail-latency tie-break; paired quality interval contains zero

Table 20: S2 streaming larger same-machine competitor check with `BAAI/bge-small-en-v1.5`, `clients_read=8`, `clients_write=2`, two repeats per engine, 1,788 matched quality queries, and 200 matched post-insert events. Paired deltas are Qdrant minus FAISS with 95% paired bootstrap intervals.

Engine / search	nDCG@10	Paired Δ nDCG@10	post_insert_hit@10,5s	Mean / max p99 (ms)	Errors
FAISS CPU / ef64	0.5055	reference	0.840	47.8 / 74.1	0
Qdrant / ef64	0.5028	-0.0027 [-0.0052, -0.0006]	0.840	17.1 / 20.7	0

Table 21: Quality floors and strict-p99 B2 survivors used by the strict-latency decision rule. Floor values come from the benchmark run configuration and match the report generator's floor rule. S2 streaming post-insert metrics are diagnostics in the decision rule, not additional hard gates.

Workload	Quality metric	Quality floor source	Floor value	Strict-p99 survivors
S1 single-hop	nDCG@10	configured quality floor	0.25	5
S2 streaming	nDCG@10 plus post-insert reporting	configured quality floor	0.25	5
S3 multi-evidence	evidence_coverage@10	configured quality floor	0.30	4

C Claim-evidence map

- Claim: MaxionBench is conformance-gated. Evidence: Table 14 reports passing conformance and behavior cards for all included engines, and the report excludes LanceDB service because conformance is missing.
- Claim: the paper contributes a reusable decision-audit protocol rather than only a benchmark implementation. Evidence: the decision-card protocol in Section 2.3 defines the conformance gate, workload evaluation, objective-specific ranking, budget-stability check, paired audit, and emitted card fields.
- Claim: strict 200 ms max-row p99 deployment selects FAISS CPU with `BAAI/bge-small-en-v1.5` for S1 single-hop, S2 streaming, and S3 multi-evidence. Evidence: Table 2 and Table 3.
- Claim: supplementary same-machine repeat evidence supports the strict FAISS CPU/bge-small decisions. Evidence: Table 8 summarizes completed local repeat rows for S1 single-hop, S2 streaming, and S3 multi-evidence.
- Claim: the selected rows are shown against the full decision surface. Evidence: Figure 2 and Table 10 show all B2 strict survivors plus objective winners outside the strict survivor set.
- Claim: strict decisions have different margins across workloads. Evidence: Table 11 shows S1 single-hop and S2 streaming as cost/quality ties resolved by latency and S3 multi-evidence as lower-cost and higher-quality than the next strict-cost candidate.
- Claim: objective choice changes deployment conclusions. Evidence: Table 3 shows different quality-first winners for all workloads and a cost-only/no-p99 shift for S3 multi-evidence.
- Claim: the S3 multi-evidence quality-first pgvector result is indistinguishable from FAISS under matched queries. Evidence: Figure 4 and Table 17 report a matched-query audit over the 5,000-query S3 multi-evidence matched-audit subset in which pgvector trails the FAISS exact FlatIP baseline by 0.0001 evidence_coverage@10 with paired 95% interval [-0.0003, 0.0000].
- Claim: the S2 streaming competitor audit finds no Qdrant quality or post-insert retrievability advantage. Evidence: Table 12 summarizes B2 post-insert diagnostics, Table 13 gives strict-choice success/miss examples and counts, and Table 20 reports a larger same-machine paired check with 1,788 matched quality observations and 200 matched post-insert events.
- Claim: budget stability is workload-dependent. Evidence: Figure 5; B0-to-B2 top-1 agreement is 0 for S1 single-hop and S2 streaming and 1 for S3 multi-evidence, with Spearman correlations 0.79, 0.43, and 0.31.

Table 22: Reviewer-facing evidence map. The main claims are framed as benchmark and decision-audit claims with stated objective and policy conditions.

Claim	Main evidence	Residual risk	Paper stance
Conformance-gated reporting	72 strict-schema run dirs; support table	Adapter coverage follows the recorded local matrix	Supported protocol claim
Strict-latency decisions	Figure 2; main and decision tables; margin table	Recorded CPU-only runtime profile	Deployment decision evidence
Strict FAISS repeat evidence	Same-machine repeat audit in Table 8	Repeats report tail-latency variation under the recorded runtime profile	Supplementary robustness check
Decision surface is visible	B2 strict survivors and objective rows in Figure 2 and Table 10	Full matrix remains appendix and supplementary material	Anti-cherry-picking evidence
S2 streaming competitor check	S2 streaming diagnostics in Tables 12–13; larger same-machine paired S2 streaming check in Table 20	Fixed 1s/5s probes and ID-level observations	Competitor quality/post-insert audit
S3 multi-evidence pgvector quality edge is indistinguishable under matched queries	5,000-query paired audit against FAISS exact FlatIP in Figure 4 and Table 17	Higher ef/probe service sweeps are extension rows	Objective-sensitivity audit
Budget stability is workload-dependent	Figure 5; B0-to-B2 top-1 and Spearman stats	Lower ranks remain noisy	Report budgets explicitly

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes].

   Justification: The abstract and introduction claim a conformance-gated local decision audit tied to a recorded experimental profile; Figure 2, Tables 2 and 3, and the appendix claim-evidence map provide the supporting evidence.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes].

   Justification: The manuscript includes an experimental-boundaries section describing the recorded CPU-only setting, S3 multi-evidence corpus, S2 streaming post-insert retrievability metric, adapter settings, and budget-stability results.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete proof?

   Answer: [N/A].

   Justification: The paper does not present theoretical results; it defines benchmark metrics and decision rules.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results?

   Answer: [Yes].

   Justification: The appendix and supplementary material describe the workflow, validation summary, runtime metadata, checksums, conformance matrix, generated reports, and consistency verifier.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results?

   Answer: [Yes].

   Justification: The benchmark code, workflow instructions, generated manifests, and frozen evaluation snapshot are part of the supplementary package prepared with this manuscript; any public release should preserve anonymization at submission time.

6. **Experimental setting/details**

   Question: Does the paper specify all training and test details necessary to understand the results?

   Answer: [Yes].

   Justification: The benchmark-design section specifies engines, datasets, embedding tiers, concurrency pins, budget levels, metrics, decision rules, and conformance gates.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about statistical significance?

Answer: [Yes].

Justification: The main table reports 95% confidence intervals; targeted S2 streaming and S3 multi-evidence audits report paired bootstrap intervals for the highest-risk margins, and Figure 4 visualizes the S3 multi-evidence paired interval.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on compute resources needed to reproduce the experiments?

Answer: [Yes].

Justification: The benchmark-design and reproducibility sections state the CPU-only runtime profile and exact budget ladder; the run metadata records per-run resource fields.

9. **Code of ethics**

Question: Does the research conducted in the paper conform with the NeurIPS Code of Ethics?

Answer: [Yes].

Justification: The work evaluates retrieval infrastructure using public benchmark artifacts and does not involve human subjects, private data, or model training on sensitive data.

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes].

Justification: The experimental-boundaries and broader-impact section discusses the benefit of transparent deployment evidence with visible quality, post-insert retrievability, objective, budget, and margin information.

11. **Safeguards**

Question: Does the paper describe safeguards for responsible release of data or models that have a high risk for misuse?

Answer: [N/A].

Justification: The paper does not release new generative models or scraped high-risk datasets; it uses existing retrieval benchmark assets.

12. **Licenses for existing assets**

Question: Are creators or original owners of assets properly credited and are licenses and terms of use explicitly mentioned and respected?

Answer: [Yes].

Justification: The manuscript cites the datasets, software systems, and embedding models used; final submission should include license manifests alongside the supplementary material.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes].

Justification: HOTPOTQA-MAXIONBENCH is documented through its preprocessing description, manifest, checksums, validation summary, and Croissant-style metadata included with the supplementary material.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include instructions, screenshots, and compensation details?

Answer: [N/A].

Justification: The paper does not involve crowdsourcing or human-subject experiments.

15. **Institutional review board approvals or equivalent for research with human subjects**

    Question: Does the paper describe risks to study participants and IRB approvals or equivalent?

    Answer: [N/A].

    Justification: The paper does not involve human-subject research.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods?

    Answer: [N/A].

    Justification: LLMs are not an important, original, or non-standard component of the benchmark method; the benchmark evaluates retrieval infrastructure and local embedding models.